

Rapport de Laboratoire – Web Mining

Anthony Atallah

March 5, 2025

Contents

1	Introduction	2
2	Partie Pratique	2
2.1	Crawling	2
2.1.1	Code du Spider (quotes_spider.py)	2
2.2	Indexation	3
2.2.1	Code d’Indexation	3
2.3	Recherche	4
2.3.1	Code de Recherche	4
3	Partie Théorique	5
3.1	Syntaxe des Requêtes Elasticsearch	5
3.2	Indexation de Pages en Plusieurs Langues	6
3.3	Recherche Floue et Alternatives	7
4	Conclusion	7

1 Introduction

Ce laboratoire a pour objectif de mettre en place une chaîne complète de traitement des données web, allant du crawling à la recherche via indexation dans Elasticsearch. Le projet se divise en quatre parties principales :

1. **Crawling:** Utilisation de la librairie Scrapy pour parcourir un site web et en extraire des données pertinentes.
2. **Indexation:** Stockage et configuration précise des champs indexés dans Elasticsearch.
3. **Recherche:** Implémentation d'une fonctionnalité de recherche interrogeant l'index et affichant les résultats avec des scores de pertinence.
4. **Questions Théoriques:** Réponses aux questions sur la syntaxe des requêtes Elasticsearch, l'indexation multilingue et la recherche floue.

2 Partie Pratique

2.1 Crawling

Nous avons développé un spider Scrapy pour parcourir le site :

<http://quotes.toscrape.com/>

Ce spider extrait les données suivantes :

- **Titre de la page** (balise <title>)
- **URL de la page**
- **Liens internes** pour permettre une exploration à une profondeur limitée (ici, profondeur 1)

2.1.1 Code du Spider (quotes_spider.py)

```
1 import scrapy
2 import json
3
4 class QuotesSpider(scrapy.Spider):
5     name = "quotes"
6     allowed_domains = ["quotes.toscrape.com"]
7     start_urls = [
8         'http://quotes.toscrape.com/'
9     ]
```

```

10
11     custom_settings = {
12         'DEPTH_LIMIT': 1,
13         'CONCURRENT_REQUESTS': 32,
14         'DOWNLOAD_DELAY': 0,
15         'LOG_LEVEL': 'INFO',
16         'HTTPCACHE_ENABLED': True,
17         'HTTPCACHE_EXPIRATION_SECS': 3600,
18     }
19
20     def parse(self, response):
21         # Extraction des données principales de la page
22         data = {
23             'url': response.url,
24             'title': response.css('title::text').get()
25         }
26         yield data
27
28         # Suivi des liens internes
29         links = response.css('a::attr(href)').getall()
30         for link in links:
31             if link.startswith('/') or "quotes.toscrape.com"
32 in link:
33                 full_url = response.urljoin(link)
34                 yield response.follow(full_url, callback=self
35 .parse)

```

Listing 1: Code du Spider

L'exécution du spider génère un fichier `output.json` contenant 48 documents, ce qui montre que nous avons parcouru plusieurs pages du site et extrait les données essentielles.

2.2 Indexation

Nous avons indexé les documents extraits dans Elasticsearch en utilisant la bibliothèque Python `elasticsearch`. Nous avons créé un index nommé `wem_index` et indexé chaque document du fichier `output.json`.

2.2.1 Code d'Indexation

```

1 import os
2 import json
3 from elasticsearch import Elasticsearch
4
5 # Connexion a Elasticsearch
6 es = Elasticsearch("http://localhost:9200")

```

```

7 INDEX_NAME = "wem_index"
8
9 # Creation de l'index si necessaire
10 if not es.indices.exists(index=INDEX_NAME):
11     es.indices.create(index=INDEX_NAME)
12     print(f"Index '{INDEX_NAME}' creer.")
13 else:
14     print(f"Index '{INDEX_NAME}' existe deja.")
15
16 # Chargement des donnees issues du crawler
17 with open("output.json", "r", encoding="utf-8") as f:
18     docs = json.load(f)
19
20 # Indexation des documents
21 for doc in docs:
22     es.index(index=INDEX_NAME, document=doc)
23 es.indices.refresh(index=INDEX_NAME)
24 print(f"Indexed {len(docs)} documents into '{INDEX_NAME}'.")

```

Listing 2: Script d'Indexation

2.3 Recherche

Nous avons développé une fonction de recherche qui interroge l'index en utilisant une requête `multi_match` qui cible plusieurs champs et applique un boost sur le champ `title` pour améliorer la pertinence des résultats.

2.3.1 Code de Recherche

```

1 from elasticsearch import Elasticsearch
2 import json
3
4 # Connexion a Elasticsearch
5 es = Elasticsearch("http://localhost:9200")
6 INDEX_NAME = "wem_index"
7
8 # Definition de la requete de recherche avec boosting sur le
  champ "title"
9 query_body = {
10     "query": {
11         "multi_match": {
12             "query": "quotes", # Terme de recherche (a
  adapter)
13             "fields": [
14                 "title^3", # Le champ "title" est booster
15                 "url"
16             ],

```

```

17         "fuzziness": "AUTO"
18     }
19 },
20     "size": 50    # Recuperer jusqu a 50 documents
21 }
22
23 response = es.search(index=INDEX_NAME, body=query_body)
24 print("Total hits:", response["hits"]["total"]["value"])
25 for hit in response["hits"]["hits"]:
26     print("\n-----")
27     print(json.dumps(hit["_source"], indent=2, ensure_ascii=
28         False))
29     print("Score:", hit["_score"])

```

Listing 3: Script de Recherche

3 Partie Théorique

3.1 Syntaxe des Requêtes Elasticsearch

Pour rechercher dans des champs spécifiques, on utilise généralement les requêtes `match` ou `multi_match`.

Requête sur un champ spécifique

```

1 {
2   "query": {
3     "match": {
4       "title": "recherche"
5     }
6   }
7 }

```

Listing 4: Requete Match sur le champ title

Requête `multi_match` sur plusieurs champs

```

1 {
2   "query": {
3     "multi_match": {
4       "query": "recherche",
5       "fields": ["title", "meta_description"]
6     }
7   }
8 }

```

```
8 }
```

Listing 5: Requete Multi_match

Boosting des champs

Le boosting permet d'accorder plus d'importance à certains champs dans le calcul de la pertinence. Par exemple :

```
1 {
2   "query": {
3     "multi_match": {
4       "query": "recherche",
5       "fields": ["title^3", "meta_description"]
6     }
7   }
8 }
```

Listing 6: Requête avec Boosting

Ici, le champ `title` est pondéré trois fois plus que `meta_description`.

3.2 Indexation de Pages en Plusieurs Langues

Pour indexer un corpus multilingue, plusieurs stratégies sont envisageables :

- **Utilisation d'analyzers spécifiques par langue :** Chaque langue a ses règles de tokenisation, de stemming et de gestion des stopwords. Dans le mapping d'Elasticsearch, il est possible d'attribuer un analyzer adapté (par exemple, `french` pour le français, `english` pour l'anglais) à chaque champ ou à des sous-champs (multi-fields).
- **Détection automatique de la langue :** Intégrer une étape de pré-traitement pour détecter la langue d'un document et indexer le contenu dans un champ dédié avec l'analyzer approprié.

Avantages :

- Une analyse linguistique adaptée améliore la précision de la recherche.

Inconvénients :

- Complexité accrue de la configuration et gestion de plusieurs analyzers.

3.3 Recherche Floue et Alternatives

La recherche floue permet de trouver des documents même si le terme recherché comporte des erreurs de frappe ou des variations. Elasticsearch utilise la distance de Levenshtein pour mesurer la proximité entre les termes.

Exemple de Recherche Floue

```
1 {  
2   "query": {  
3     "match": {  
4       "title": {  
5         "query": "caitlin",  
6         "fuzziness": "AUTO"  
7       }  
8     }  
9   }  
10 }
```

Listing 7: Recherche Floue

Problématique : Pour des prénoms comportant de nombreuses variations (ex. Caitlin, Caitlyn, etc.), la recherche floue peut être coûteuse en termes de performance.

Alternatives :

- **Analyzers phonétiques :** Utiliser des plugins tels que `elasticsearch-analysis-phonetic` pour convertir les termes en une représentation phonétique et ainsi regrouper les variations similaires.
- **Normalisation des données :** Prétraiter les textes (minuscule, suppression des accents, etc.) pour améliorer les correspondances exactes.
- **Combinaison des méthodes :** Utiliser une recherche exacte sur des champs normalisés en parallèle d'une recherche floue sur des champs dédiés.

4 Conclusion

Ce laboratoire a permis de mettre en place une chaîne complète allant du crawling de pages web à leur indexation dans Elasticsearch, puis à la recherche avec un système de boosting et de recherche floue pour améliorer la pertinence des résultats. Les aspects théoriques abordent les mécanismes des requêtes Elasticsearch, les stratégies pour l'indexation multilingue et les alternatives à

la recherche floue. Ces travaux fournissent une base solide pour une solution de Web Mining et peuvent être étendus selon les besoins futurs.